Implementation and Validation of Post-Quantum Cryptography on **STM32F4 Discovery** Microcontroller

Date: 23.02.2026

Name: Rishi Raj Gautam

Mtr. No: 5008056

INTRODUCTION

- Implementing post-quantum cryptography (PQC) [1] on a resource-constrained embedded device
- Using the pqm4 [2] benchmarking framework
- Running ML-KEM-768 [3] on an STM32F4 Discovery board [4]
- Performing Known Answer Test (KAT) validation against official NIST test vectors

MOTIVATION

Modern cryptographic systems (RSA, ECC) can have a future threat of quantum computers. For example:

- Shor's algorithm [5] tends to break RSA and ECC
- Grover's algorithm [6] weakens symmetric crypto

Post-quantum cryptography (PQC) aims to provide quantum-resistant algorithms.

- **PQC design goals**
  - Quantum resistance
  - Practical performance
  - Standardization by NIST

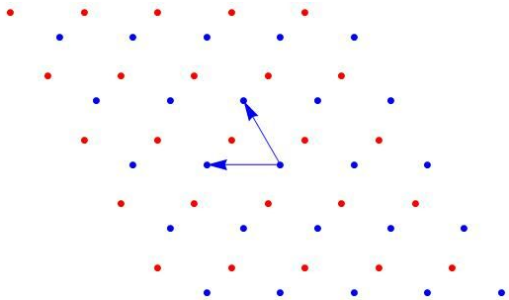

Fig 1: Vector Space

OBJECTIVE

- **Primary objectives**
  - Deploy post-quantum cryptography on an embedded platform
  - Validate correctness using Known Answer Tests (KATs)

- **Future objectives**
  - Understand performance and resource constraints
  - Try to attack the PQC algorithms via Side-Channel Analysis Methods [7]

BACKGROUND

- PQM4
  - A benchmarking and testing framework for post-quantum crypto
  - Designed for ARM Cortex-M4 microcontrollers
  - Developed by the **MUPQ** project [2]
  - Widely used in PQC research and standardization evaluations

- Key features:
  - Supports KEMs and Digital Signatures
  - Measures:
    - Execution speed
    - Stack usage
    - Hashing cost

- Runs on real hardware (STM32, Nucleo, etc.)

BACKGROUND

- ML-KEM
  - Stands for Module-Lattice Key Encapsulation Mechanism
  - Standardized as FIPS 203 by NIST [3]
  - Based on lattice-based cryptography
  - Successor of the Crystals-Kyber algorithm [8]

- Purpose of a KEM:
  - Securely establish a shared secret
  - Used to derive symmetric keys (e.g., AES keys)
  - Used in TLS, VPNs, secure messaging

BACKGROUND

Why ML-KEM-768?

NIST has defined three parameter sets:

Variant	Security	Performance	Equivalent "Pre-Quantum" Strength
ML-KEM-512	Lower	Faster	AES-128
ML-KEM-768	Balanced	Balanced	AES-192
ML-KEM-1024	Highest	Slowest	AES-256

ML-KEM-768:

- Recommended default security level
- Good balance for embedded systems
- Widely used in real deployments and benchmarks

METHODOLOGY

- Hardware Setup
- Software Setup
- Regular workflow
  - Compile code
  - Flash binary
  - Capture output
- Validation against KAT

Fig 2: STM32 Discovery Board

METHODOLOGY

Hardware Setup

- Hardware used:
  - STM32F4 Discovery Board
  - USB-to-UART adapter
  - Jumper wires

- UART wiring:
  - PA2 → RX | PA3 → TX
  - GND → GND

- Purpose of UART:
  - Capture cryptographic outputs
  - Read benchmark results
  - Validate test vectors

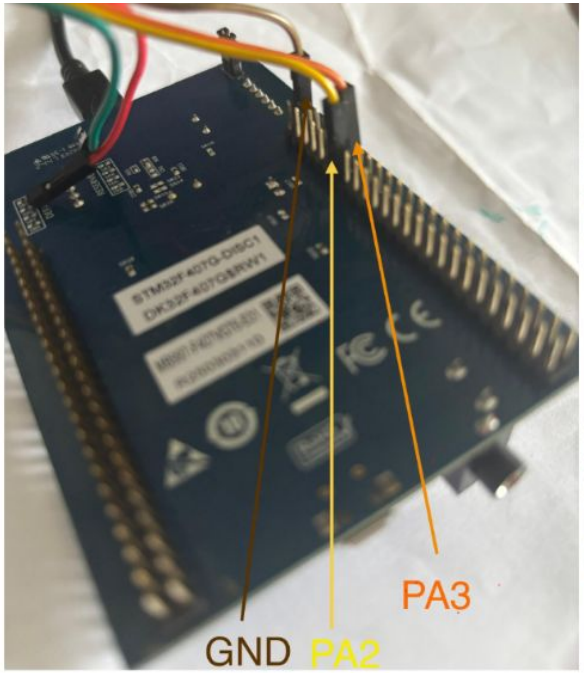

Fig 3: UART Wiring

Software Setup

Toolchain:

- ARM GNU Embedded toolchain (arm-none-eabi-gcc)
- make, git
- stlink for flashing
- serial output receiver

METHODOLOGY

Regular workflow (High-Level)

- Set up Hardware and Software toolchain
- Build PQM4 for STM32F4
- Capture serial output
- Validate Output Values

Observed Outputs :

```
DONE key pair generation! DONE
encapsulation! DONE
decapsulation! OK KEYS

==========================
keypair cycles: 622953
keypair hash cycles: 457101
encaps cycles: 641570
encaps hash cycles: 468150
decaps cycles: 688734
decaps hash cycles: 468068
OK KEYS
```

METHODOLOGY

Validation against KAT

- Implement deterministic RNG (DRBG) [13]
  - Implement the DRBG used by NIST KAT generation
  - Seeded using test vectors from **prompt.json** [9,10] files
  - Ensures reproducible, exact outputs from **expectedResults.json** [11,12]


```
{
  "tgId": 2,
  "testType": "AFT",
  "parameterSet": "ML-KEM-768",
  "function": "encapsulation",
  "tests": [
    {
      "tcId": 26,
      "ek": "F255CE47334283B8622BE7C
      "m": "5BD922AF345AB90F297D0A82
    },
```

```
{
  "tgId": 2,
  "testType": "AFT",
  "parameterSet": "ML-KEM-768",
  "tests": [
    {
      "tcId": 26,
      "z": "D6BF055CB7B375E3271E
      "d": "A2B4BCA315A6EA4600B4A
    },
```


Fig4: prompt.json


```
{
  "tgId": 2,
  "tests": [
    {
      "tcId": 26,
      "c": "4EE24D9E0858B36DC755A9
      "k": "B2425299020BCF563B8EBE
    },
```

```
{
  "tgId": 2,
  "tests": [
    {
      "tcId": 26,
      "ek": "5219C4CC17
      "dk": "E6F634F0A3
    },
```


Fig5: expectedResults.json

Results

Output Binaries:

pqm4 builds multiple binaries per algorithm:

- ***_test.elf** → functional correctness checks
- ***_speed.elf** → cycle count measurements
- ***_hashing.elf** → hashing cost
- ***_stack.elf** → stack usage
- ***_testvectors.elf** → deterministic output

Each binary focuses on one specific evaluation

Results

Observed results:

- ML-KEM-768 successfully runs on STM32F4
- Encapsulated and decapsulated shared secrets matched
- Deterministic outputs achieved using DRBG
- Output structure matches NIST reference format

Challenges

- Toolchain and build complexity
- UART debugging
- Deterministic randomness implementation

Conclusion

What was achieved

- Successfully deployed post-quantum cryptography on embedded hardware
- Validated ML-KEM using deterministic KAT methodology
- Gained insight into performance and correctness constraints
- Ready for Side-Channel Analysis Measurements.

Future Works

Possible extensions

- Benchmark performance across parameter sets (512 vs 768 vs 1024)
- Compare different PQC algorithms
- Implement PQC using other libraries.
- SIde-Channel Attack analysis

References

1. https://csrc.nist.gov/Projects/post-quantum-cryptography
2. https://github.com/mupq/pqm4
3. https://csrc.nist.gov/pubs/fips/203/final
4. https://www.st.com/resource/en/schematic_pack/mb997-f407vgt6-e01_schematic.pdf
5. https://arxiv.org/abs/quant-ph/9508027
6. https://arxiv.org/abs/quant-ph/9605043
7. https://csrc.nist.gov/csrc/media/events/non-invasive-attack-testing-workshop/documents/08_goodwill.pdf
8. https://pq-crystals.org/kyber/
9. https://github.com/usnistgov/ACVP-Server/blob/master/gen-val/json-files/ML-KEM-keyGen-FIPS203/prompt.json
10. https://github.com/usnistgov/ACVP-Server/blob/master/gen-val/json-files/ML-KEM-encapDecap-FIPS203/prompt.json
11. https://github.com/usnistgov/ACVP-Server/blob/master/gen-val/json-files/ML-KEM-keyGen-FIPS203/expectedResults.json
12. https://github.com/usnistgov/ACVP-Server/blob/master/gen-val/json-files/ML-KEM-encapDecap-FIPS203/expectedResults.json
13. https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/archive/api-notes-sept2017.pdf?utm_source=chatgpt.com